

Mreža električnih punionica u gradu Zagrebu

1. Uvod

Električna vozila posljednjih godina postaju sve važniji dio modernog prometa, a njihov razvoj prati i potreba za kvalitetnom infrastrukturom električnih punionica. Kako broj punionica raste, postaje važno omogućiti jednostavno pronalaženje najbliže i odgovarajuće punionice te analizirati povezanost cijele mreže unutar grada.

Zbog velikog broja međusobnih odnosa između punionica, poput njihove lokacije, udaljenosti i tipova priključaka, grafovske baze podataka predstavljaju vrlo prikladan način modeliranja takvih podataka.

U ovom projektu izrađena je grafovska baza podataka pod nazivom „Mreža električnih punionica u gradu Zagrebu“ korištenjem Neo4j baze podataka i Cypher jezika. Projekt se temelji na stvarnim podacima s Portala otvorenih podataka Grada Zagreba.

Cilj projekta bio je prikazati mrežu električnih punionica u obliku grafa te demonstrirati mogućnosti grafovskih baza podataka kroz funkcionalnosti poput pronalaženja najbliže kompatibilne punionice, analize povezanosti mreže i pronalaženja najkraćeg puta između punionica.

2. Opis skupa podataka

Za izradu projekta korišten je javno dostupan skup podataka https://opendata.arcgis.com/api/v3/datasets/4a4fc728724b4d319c27a9f647a0bb62_0/downloads/data?format=csv&spatialRefId=4326&where=1%3D1 s Portala otvorenih podataka Grada Zagreba. Skup podataka sadrži informacije o električnim punionicama na području grada Zagreba.

Podaci uključuju:

- naziv punionice,
- adresu,
- geografske koordinate,
- broj utičnica,
- tip utičnice.

U projektu je korišteno ukupno 40 električnih punionica.

3. Grafovska baza podataka i korištene tehnologije

Za izradu projekta korištena je grafovska baza podataka Neo4j AuraDB. Neo4j koristi property graph model, u kojem se podaci prikazuju pomoću čvorova, veza i njihovih svojstava.

U projektu je korišten i Cypher, jezik namijenjen radu s grafovskim bazama podataka. Cypher omogućuje jednostavno pretraživanje i analizu grafova korištenjem obrazaca čvorova i relacija.

Grafovski model pokazao se prikladnim za ovaj projekt jer omogućuje:

- prirodan prikaz mreže električnih punionica,
- jednostavno povezivanje punionica prema udaljenosti,
- analizu povezanosti mreže,
- pronalaženje najbližih punionica,
- izvođenje shortest path algoritama između punionica.

4. Model podataka

Grafovski model baze podataka sastoji se od čvorova i relacija koje predstavljaju mrežu električnih punionica u Gradu Zagrebu.

4.1 Čvor Punionica

Glavni čvor u modelu je Punionica, koji predstavlja jednu električnu punionicu.

Svaka punionica sadrži sljedeća svojstva:

- id – jedinstveni identifikator,
- naziv – naziv punionice,
- adresa – adresa punionice,
- lat – geografska širina,
- lon – geografska dužina,
- broj_uticnica – broj utičnica na punionici.

Primjer čvora:

```
(:Punionica {  
  id: 1,  
  naziv: 'HR050024',  
  adresa: 'JANKOMIR 33',  
  lat: 45.798742286,  
  lon: 15.885127074,  
  broj_uticnica: 2  
})
```

4.2 Čvor TipUticnice

Čvor TipUticnice predstavlja tip priključka koji određena punionica podržava.

Svojstvo čvora:

- naziv – naziv tipa utičnice.

Primjeri:

```
(:TipUticnice {naziv: 'AC'})  
(:TipUticnice {naziv: 'DC'})
```

4.3 Relacija IMA_TIP

Relacija IMA_TIP povezuje punionicu s tipom utičnice koji podržava.

Primjer:

```
(:Punionica)-[:IMA_TIP]->(:TipUticnice)
```

Jedna punionica može biti povezana s više tipova utičnica.

4.4 Relacija POVEZANA_S

Relacija POVEZANA_S povezuje dvije punionice koje se nalaze u međusobnoj blizini.

Relacija sadrži svojstvo:

- udaljenost_km – udaljenost između dvije punionice izražena u kilometrima.

Primjer:

```
(:Punionica)-[:POVEZANA_S {udaljenost_km: 2.14}]->(:Punionica)
```

Dvije punionice povezane su ako je udaljenost između njih manja ili jednaka 3 kilometra.

5. Implementacija baze podataka

Baza podataka izrađena je u Neo4j AuraDB sustavu korištenjem Cypher jezika. Prvo je postavljeno ograničenje jedinstvenosti kako bi svaka punionica imala jedinstveni identifikator, a svaki tip utičnice jedinstveni naziv.

Podaci su zatim učitani iz CSV datoteke pomoću naredbe `LOAD CSV WITH HEADERS`.

Za kreiranje relacija između punionica i tipova utičnica korištena je funkcija `split()`, kojom su vrijednosti poput „AC, DC” razdvojene na zasebne tipove punjenja. Nakon toga kreirani su čvorovi `TipUticnice` i relacije `IMA_TIP` između punionica i odgovarajućih tipova utičnica.

Primjer kreiranja relacije `IMA_TIP`:

```
MATCH (p:Punionica)
  UNWIND split(p.tip_uticnice, ',') AS tip
  WITH p, trim(tip) AS tipNaziv
  WHERE tipNaziv <> ''
  MERGE (t:TipUticnice {naziv: tipNaziv})
  SET t.name = tipNaziv
  MERGE (p)-[:IMA_TIP]->(t);
```

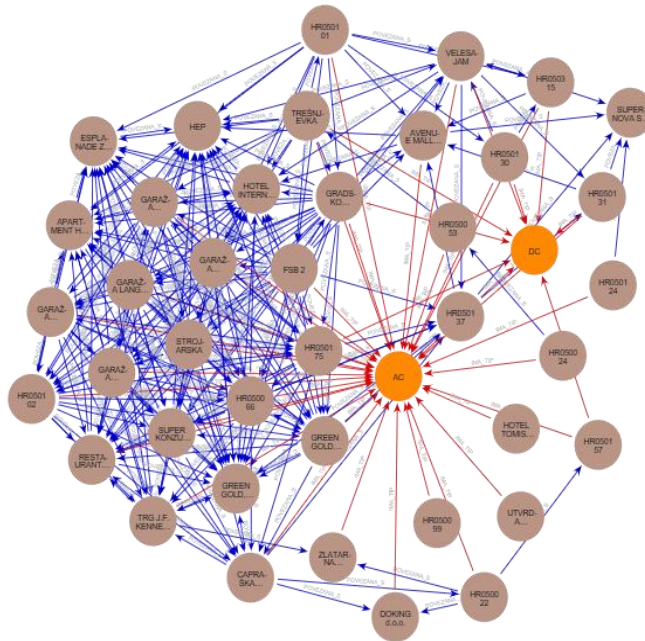
Kako bi se stvorila mreža punionica, između prostorno bliskih punionica dodane su relacije `POVEZANA_S`. Udaljenosti su izračunate pomoću funkcije `point.distance()`.

Primjer povezivanja punionica:

```
MATCH (a:Punionica), (b:Punionica)
  WHERE a.id < b.id
  WITH a, b,
  point.distance(
    point({latitude: a.lat, longitude: a.lon}),
    point({latitude: b.lat, longitude: b.lon})
  ) / 1000 AS udaljenost_km
  WHERE udaljenost_km <= 3
  MERGE (a)-[r:POVEZANA_S]->(b)
  SET r.udaljenost_km = round(udaljenost_km * 100) / 100;
```

Na ovaj način stvorena je mreža od 40 punionica povezanih prostornim relacijama.

6. Prikaz gotovog grafa



Na slici je prikazan grafovski model mreže električnih punionica. Čvorovi predstavljaju punionice i tipove utičnica, dok relacije prikazuju podržane tipove punjenja i prostornu povezanost između punionica.

7. Cypher upiti i analiza podataka

Nakon izrade baze podataka nad grafom su izvršeni različiti Cypher upiti kako bi se analizirala mreža električnih punionica i demonstrirale mogućnosti grafovskih baza podataka.

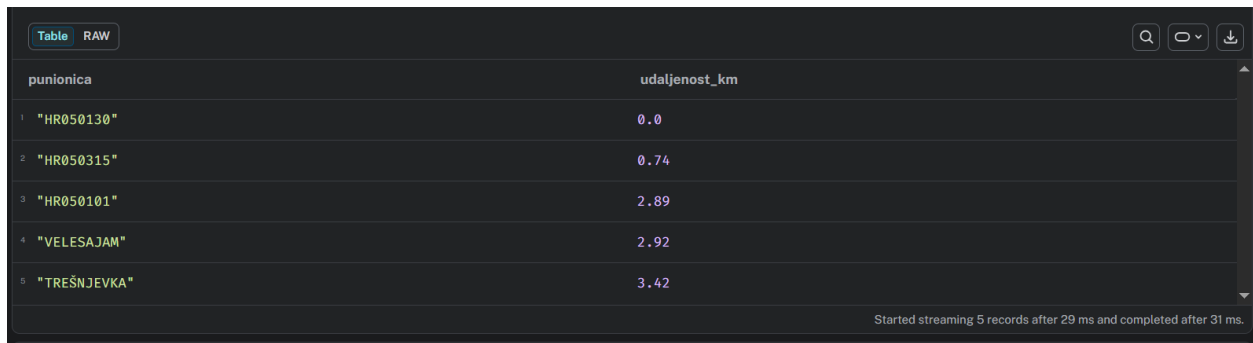
7.1 Pronalaženje najbližih punionica

Sljedeći upit pronalazi najbliže punionice zadanoj lokaciji:

```
MATCH (p:Punionica)
WITH p,
point.distance(
  point({latitude: 45.768984481, longitude: 15.934266084}),
  point({latitude: p.lat, longitude: p.lon})
) / 1000 AS udaljenost_km
RETURN p.naziv AS punionica,
       round(udaljenost_km * 100) / 100 AS udaljenost_km
```

```
ORDER BY udaljenost_km  
LIMIT 5;
```

Ovim upitom moguće je brzo pronaći najbliže punionice korisniku.



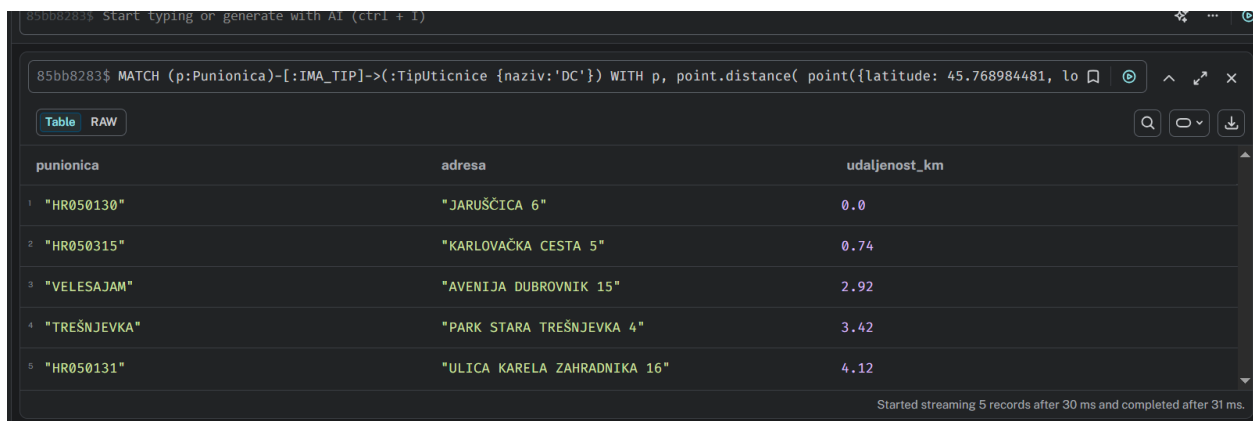
punionica	udaljenost_km
"HR050130"	0.0
"HR050315"	0.74
"HR050101"	2.89
"VELESAJAM"	2.92
"TREŠNJEVKA"	3.42

Started streaming 5 records after 29 ms and completed after 31 ms.

7.2 Pronalaženje najbližihDC punionica

```
MATCH (p:Punionica)-[:IMA_TIP]->(:TipUticnice {naziv:'DC'})  
WITH p,  
point.distance(  
  point({latitude: 45.768984481, longitude: 15.934266084}),  
  point({latitude: p.lat, longitude: p.lon})  
) / 1000 AS udaljenost_km  
RETURN p.naziv AS punionica,  
       p.adresa AS adresa,  
       round(udaljenost_km * 100) / 100 AS udaljenost_km  
ORDER BY udaljenost_km  
LIMIT 5;
```

Upit pronalazi najbliže punionice koje podržavaju DC punjenje u odnosu na zadanu lokaciju korisnika.



punionica	adresa	udaljenost_km
"HR050130"	"JARUŠČICA 6"	0.0
"HR050315"	"KARLOVAČKA CESTA 5"	0.74
"VELESAJAM"	"AVENIJA DUBROVNIK 15"	2.92
"TREŠNJEVKA"	"PARK STARA TREŠNJEVKA 4"	3.42
"HR050131"	"ULICA KARELA ZAHRAĐNIKA 16"	4.12

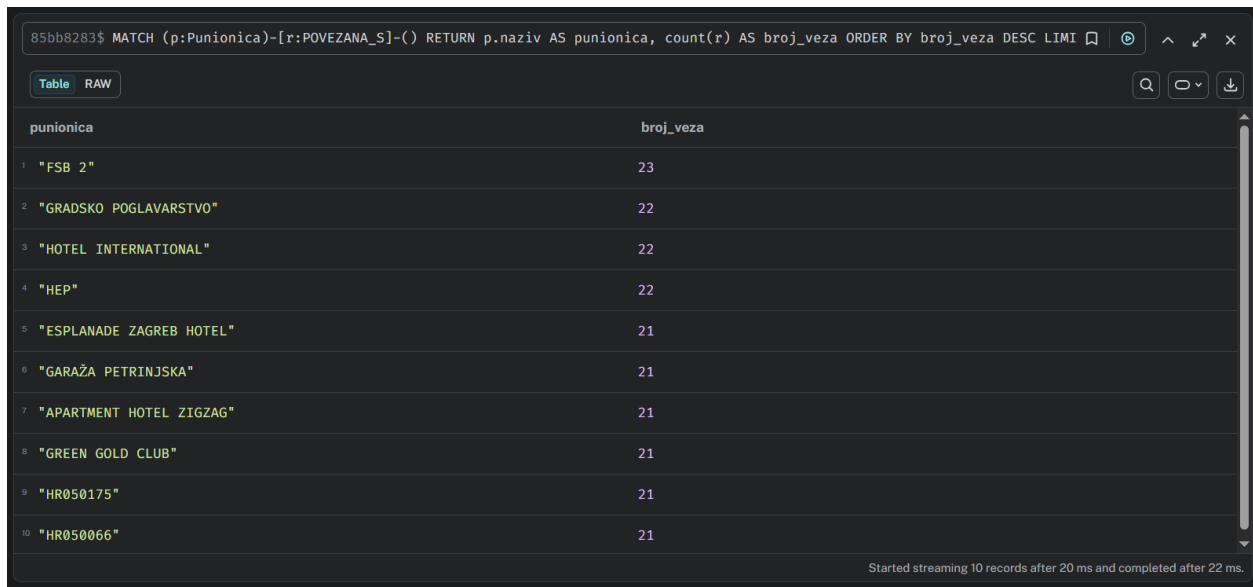
Started streaming 5 records after 30 ms and completed after 31 ms.

7.3 Analiza povezanosti mreže

```
MATCH (p:Punionica)-[r:POVEZANA_S]-()  
RETURN p.naziv AS punionica,  
       count(r) AS broj_veza
```

```
ORDER BY broj_veza DESC  
LIMIT 10;
```

Ovim upitom moguće je pronaći najpovezanije punionice u mreži.



The screenshot shows a Neo4j Cypher query interface. The query is: `MATCH (p:Punionica)-[r:POVEZANA_S]-() RETURN p.naziv AS punionica, count(r) AS broj_veza ORDER BY broj_veza DESC LIMIT 10`. The results are displayed in a table with two columns: 'punionica' and 'broj_veza'. The table lists the top 10 most connected bus stops.

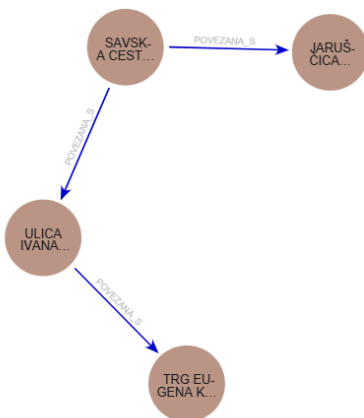
	punionica	broj_veza
1	"FSB 2"	23
2	"GRADSKO POGLAVARSTVO"	22
3	"HOTEL INTERNATIONAL"	22
4	"HEP"	22
5	"ESPLANADE ZAGREB HOTEL"	21
6	"GARAŽA PETRINJSKA"	21
7	"APARTMENT HOTEL ZIGZAG"	21
8	"GREEN GOLD CLUB"	21
9	"HR050175"	21
10	"HR050066"	21

Started streaming 10 records after 20 ms and completed after 22 ms.

7.4 Pronalaženje najkraćeg puta

```
MATCH (start:Punionica {naziv:'HR050130'}),  
      (cilj:Punionica {naziv:'GARAŽA KVATRIČ'})  
MATCH p = shortestPath((start)-[:POVEZANA_S*]-(cilj))  
RETURN p;
```

Ovim upitom moguće je pronaći najkraći povezani put između dvije punionice u mreži. Takva analiza može biti korisna kod planiranja optimalnih ruta električnih vozila i prikazuje jednu od glavnih prednosti grafovskih baza podataka.



8. Zaključak

U ovom projektu izrađena je grafovska baza podataka koja prikazuje mrežu električnih punionica na području Grada Zagreba korištenjem Neo4j baze podataka i Cypher jezika. Podaci su modelirani pomoću čvorova i relacija, čime je omogućeno prirodno prikazivanje povezanosti između punionica.

Kroz projekt su implementirane funkcionalnosti poput pronalaženja najbližih i kompatibilnih punionica, analize povezanosti mreže te pronalaženja najkraćeg puta između punionica. Na taj način prikazane su glavne prednosti grafovskih baza podataka kod rada s međusobno povezanim podacima i mrežnim analizama.

Projekt pokazuje da su grafovske baze podataka vrlo prikladne za sustave koji uključuju prostorne odnose, mreže i traversal algoritme te predstavljaju dobro rješenje za razvoj pametnih gradskih sustava i moderne prometne infrastrukture.